

CHAPTER 6:

INHERITANCE

Definition

Relationship among classes where one class shares the structure and behavior defined in one or more other classes.

- ✓ software reusability
- ✓relationship between a superclass and its subclasses
- ✓used to build new classes from existing classes.
  - Absorb existing class's data and behaviors
  - Enhance with new capabilities

Superclass & Subclass

- ✓A class that is derived from another class is called a *subclass* (also a *derived class*, *extended class*, or *child class*).
- ✓The class from which the subclass is derived is called a *superclass* (also a *base class* or a *parent class*).
- ✓The inheritance relationship is transitive: if class x extends class y, then a class z, which extends class x, will also inherit from class y.

Superclass & Subclass

- ✓A subclass inherits all the *members* (fields, methods, and nested classes) from its superclass.
- ✓Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.

- ✓ extend a class by using the extends keyword in a class declaration,

The Parent class

```
public class Parent {
    }
```

The Child class

```
public class Child extends Parent {
    }
```

Superclass & Subclass

Inheritance Examples

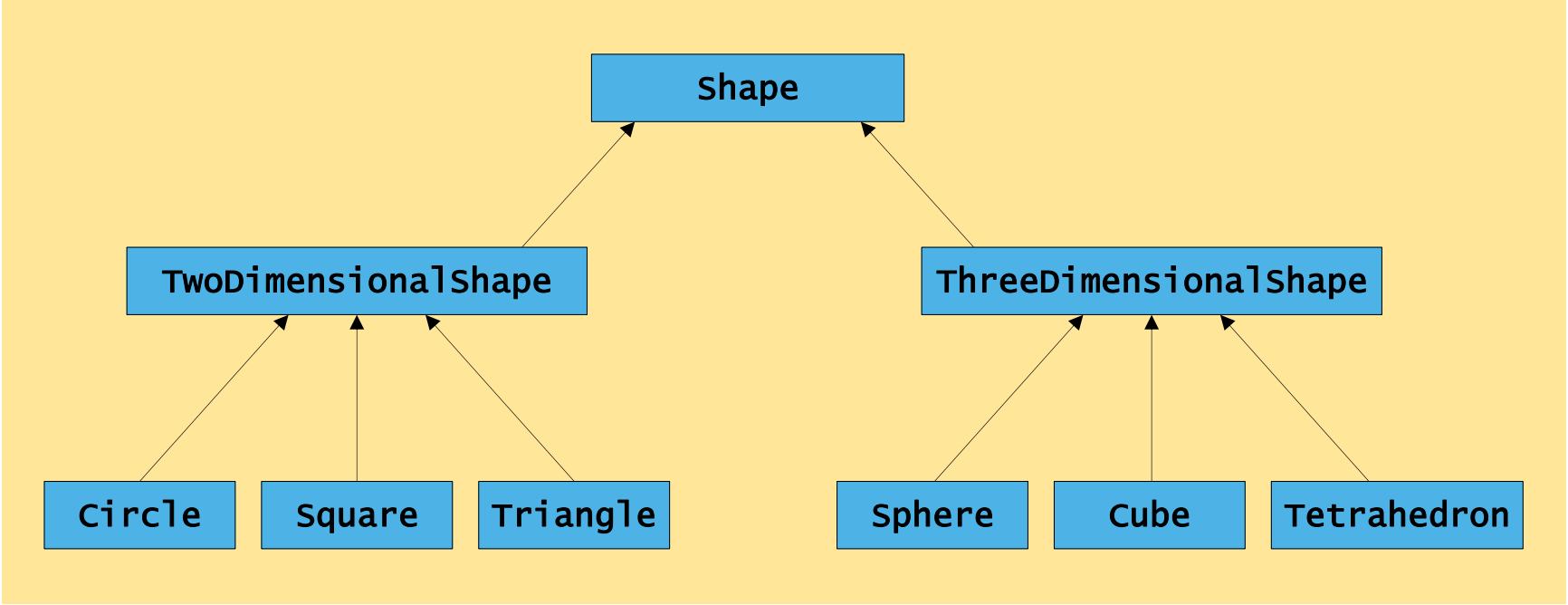
Superclass	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

Superclass & Subclass

```
public class MountainBike extends Bicycle {
// the MountainBike subclass adds one field
    public int seatHeight;
// the MountainBike subclass has one constructor
    public MountainBike(int startHeight, int startCadence, int
        startSpeed, int startGear) {
        super(startCadence, startSpeed, startGear);
         seatHeight = startHeight; }
// the MountainBike subclass adds one method
    public void setHeight(int newValue) {
        seatHeight = newValue; }
}
```

Superclass & Subclass

Inheritance Hierarchy



Method Overriding

- ✓Method Overriding is achieved when a subclass overrides non-static methods defined in the superclass, following which the new method implementation in the subclass that is executed.

- ✓The new method definition must have the same method signature (i.e., method name and parameters) and return type. Only parameter types and return type are chosen as criteria for matching method signature

```
public class Animal {
    public static void testClassMethod() {
        System.out.println("The class method in Animal."); }
    public void testInstanceMethod() {
        System.out.println("The instance method in Animal."); } }

class Cat extends Animal {
    public static void testClassMethod() {
        System.out.println("The class method in Cat."); }
    public void testInstanceMethod() {
        System.out.println("The instance method in Cat."); }
    public static void main(String[] args) {
        Cat myCat = new Cat();
        Animal myAnimal = myCat;
        Animal.testClassMethod();
        myAnimal.testInstanceMethod(); }
}
```

Accessing superclass members

- ✓If a method overrides one of its superclass's methods, we can invoke the overridden method through the use of the keyword **super**.
- ✓Consider this class, Superclass:

```
public class Superclass {
    public void printMethod() {
        System.out.println("Printed in Superclass."); } }
```

Here is a subclass, called Subclass, that overrides printMethod():

```
public class Subclass extends Superclass {
  public void printMethod() { //overrides printMethod in Superclass
     super.printMethod();
     System.out.println("Printed in Subclass"); }
  public static void main(String[] args) {
     Subclass s = new Subclass(); s.printMethod(); }
}
```

Calling superclass constructor

✓use the super keyword to invoke a superclass's constructor.

✓ Recall from the Bicycle example that MountainBike is a subclass of Bicycle.

✓ Here is the MountainBike (subclass) constructor that calls the superclass constructor and then adds initialization code of its own:

```
public MountainBike(int startHeight, int startCadence, int startSpeed, int startGear) {
    super(startCadence, startSpeed, startGear); seatHeight = startHeight; }
```

✓ Invocation of a superclass constructor must be the first line in the subclass constructor.

✓ The syntax for calling a superclass constructor is **super();** --or--
super(parameter list);

✓ With super(), the superclass no-argument constructor is called.

✓ With super(parameter list), the superclass constructor with a matching parameter list is called.

Lab Task

1. Create a Java program on simple calculation eg. Arithmetic operation applying inheritance.